

CS202 现场加指令 · 完整教程（练习题 + 流程手册）

本文档分两部分：**Part A** 练习题（12 道，自测用），**Part B** 考试现场完整流程手册。

Part A：练习题（12 道）

常踩的坑：- 有符号比较用 `sa/sb`，无符号用 `a/b` - ALU 里 `a=rs1`，`b=rs2`，别写反 - 新 opcode 在 `alu_ctrl` 不加映射 → 默认变 ADD - Load/Store 用 `funct3` 区分别依赖 `mem_size`（只 2 位会撞码）

题目

第 1 题 · R 型 MAX（取较大值，有符号）

- opcode= 0110011 · funct3= 000 · funct7= 0000011
- 语义： `rd = max(rs1, rs2)`（有符号）

第 2 题 · R 型 ANDN（位清除）

- opcode= 0110011 · funct3= 000 · funct7= 0000100
- 语义： `rd = rs1 & ~rs2`

第 3 题 · 新分支 BGT（有符号大于则跳）

- opcode= 1100011 · funct3= 010（保留没用的，现在是 NOP，要激活）
- 语义： `if (rs1 > rs2 有符号) PC += imm`

第 4 题 · Store 变体 SWU（存之前取反）

- opcode= 0100011 · funct3= 011（保留没用的）
- 语义： `mem[rs1+offset] = ~rs2`（整字取反后存）

第 5 题 · Load 变体 LWN（读回取反）

- opcode= 0000011 · funct3= 011（保留没用的）
- 语义： `rd = ~mem[rs1+offset]`（读一个字取反写 rd）

第 6 题 · 新 opcode LUI2 (U 型立即数)

- opcode= 0011011 (自定义, 非标准) • U 型
- 语义: $rd = imm \ll 12$ (同 LUI, 但新 opcode)

第 7 题 · I 型 SUBI (立即数减法)

- opcode= 0001011 (自定义) • funct3= 000 • I 型
- 语义: $rd = rs1 - imm$ (RISC-V 没有 SUBI, 真新指令)

第 8 题 · 全新立即数格式 LI16 (最难, imm_gen 都要改)

- opcode= 0101011 (自定义) • 新立即数格式
- 语义: $rd = \{16'b0, imm16\}$, 其中 $imm16 = inst[31:16]$ (16 位零扩展)
- 指令布局: $inst[31:16]=imm16$, $inst[15:12]=0$, $inst[11:7]=rd$, $inst[6:0]=opcode$

第 9 题 · ABS (绝对值, 一个源操作数)

- opcode= 0110011 • funct3= 000 • funct7= 0001000
- 语义: $rd = |rs1|$ (有符号绝对值, **rs2 不用**)

```
mem[0] = 32'hFF900293 // ADDI x5, x0, -7
mem[1] = 32'h10028333 // ABS  x6, x5      (rs2 填 x0)
mem[2] = 32'h0000006F // JAL x0, 0
```

预期: $x6 = 7$ 。练什么: 只读一个源寄存器的指令。rs2 字段填 x0 (不用), ALU 里 b 也用不上。提示: $result = sa[31] ? (\sim a + 1) : a$; (符号位为 1 就取负)

第 10 题 · ROR (循环右移)

- opcode= 0110011 • funct3= 000 • funct7= 0001001
- 语义: $rd = rs1$ 循环右移 $rs2[4:0]$ 位 (移出去的位从高位转回来)

```
mem[0] = 32'hFF900293 // ADDI x5, x0, -7 (0xFFFFFFFF9)
mem[1] = 32'h00200393 // ADDI x7, x0, 2
mem[2] = 32'h12728433 // ROR  x8, x5, x7
mem[3] = 32'h0000006F // JAL x0, 0
```

预期: $x8 = 0x7FFFFFFE$ ($0xFFFFFFFF9$ 循环右移 2)。提示: $result = (a \gg b[4:0]) | (a \ll (32 - b[4:0]))$; (右移的部分 OR 上转回来的部分)

第 11 题 · CLZ (前导零计数, 一个源操作数)

- opcode= 0110011 • funct3= 000 • funct7= 0001010

- 语义: `rd = rs1` 最高位起连续 0 的个数 (`rs2` 不用; 全 0 则结果 32)

```
mem[0] = 32'h00200393 // ADDI x7, x0, 2    (0x2 = ...0010)
mem[1] = 32'h140384B3 // CLZ  x9, x7      (rs2 填 x0)
mem[2] = 32'h0000006F // JAL  x0, 0
```

预期: `x9 = 30` (`0x2` 前面有 30 个 0)。练什么: 优先编码器逻辑 (组合逻辑稍长, 但不碰数据通路)。提示: 用嵌套三元运算从高位往低位查第一个 1: `a[31]?0 : a[30]?1 : ... : a[0]?31 : 32` (32 个分支, 全 0 返回 32)

第 12 题 • SEQZ (判零置位, 一个源操作数)

- opcode= 0110011 • funct3= 000 • funct7= 0001011
- 语义: `rd = (rs1 == 0) ? 1 : 0` (`rs2` 不用)

```
mem[0] = 32'h00000313 // ADDI x6, x0, 0
mem[1] = 32'h16030633 // SEQZ x12, x6    (0==0 → 1)
mem[2] = 32'h0000006F // JAL  x0, 0
```

预期: `x12 = 1`。提示: `result = (a == 32'b0) ? 32'd1 : 32'd0;`

答案区

第 1 题 MAX

- **alu.v:** localparam OP_MAX = 4'b1010; , case 加 OP_MAX : result = (sa > sb) ? a : b;
- **alu_ctrl.v:** R 型 funct3=000 的 funct7 case 加 7'b0000011: alu_op = OP_MAX;
- **ctrl.v:** R 型 funct3=000 的 if 链加 || funct7==7'b0000011

第 2 题 ANDN

- **alu.v:** OP_ANDN=4'b1100; case 加 OP_ANDN : result = a & ~b;
- **alu_ctrl.v:** funct3=000 funct7 case 加 7'b0000100: alu_op = OP_ANDN;
- **ctrl.v:** funct3=000 if 链加 || funct7==7'b0000100

第 3 题 BGT

- **alu.v:** OP_BGT=4'b1111; case 加 OP_BGT : result = (sa > sb) ? 32'd1 : 32'd0;
- **alu_ctrl.v:** Branch case 加 3'b010: alu_op = OP_BGT;
- **ctrl.v:** Branch 合法列表加 3'b010
- **cpu.v:** take_branch case 加 3'b010: take_branch = alu_result[0];

第 4 题 SWU

- **ctrl.v:** Store 合法列表加 3'b011

- **load_store_fmt.v:** Store 段 case(mem_size) 加 2'b11: begin mem_wdata = ~rs2_data; byte_en = 4'b1111; end (或用 funct3 端口: if(funct3==3'b011) ... , 更稳)

第 5 题 LWN

- **ctrl.v:** Load 合法列表加 3'b011
- **load_store_fmt.v:** Load 段 case(mem_size) 加 2'b11: load_data = ~mem_rdata; (或用 funct3 端口: if(funct3==3'b011) load_data = ~mem_rdata; else ...)

第 6 题 LUI2

- **ctrl.v:** 加 case 7'b0011011: begin alu_src_a = 2'b10; alu_src_b = 1'b1; wb_src = 2'b00; reg_write = 1'b1; imm_type = 3'b011; end

第 7 题 SUBI

- **ctrl.v:** 加 case 7'b0001011: begin alu_src_a = 2'b00; alu_src_b = 1'b1; wb_src = 2'b00; reg_write = 1'b1; imm_type = 3'b000; end
- **alu_ctrl.v:** case(opcode) 加 7'b0001011: alu_op = OP_SUB;

第 8 题 LI16

- **imm_gen.v:** 加 wire [31:0] imm_x = {16'b0, inst[31:16]};, case 加 3'b101: imm = imm_x;
- **ctrl.v:** 加 case 7'b0101011: begin alu_src_a = 2'b10; alu_src_b = 1'b1; wb_src = 2'b00; reg_write = 1'b1; imm_type = 3'b101; end

第 9-12 题都是 R 型 (opcode=0110011, funct3=000), 共用现成 R 路径, **ctrl.v** 只需在 **funct3=000** 的 if 链补一串 **funct7** (一次补齐 4 个): if (funct7==FUNCT7_STD || funct7==FUNCT7_ALT || funct7==FUNCT7_MUL || funct7==7'b0001000 || funct7==7'b0001001 || funct7==7'b0001010 || funct7==7'b0001011) begin ... end alu.v 先声明编码: localparam OP_ABS=4'b1010, OP_ROR=4'b1100, OP_CLZ=4'b1101, OP_SEQZ=4'b1110;

第 9 题 ABS

- **alu.v:** case 加 OP_ABS : result = sa[31] ? (~a + 1) : a;
- **alu_ctrl.v:** funct3=000 funct7 case 加 7'b0001000: alu_op = OP_ABS;
- **ctrl.v:** funct3=000 if 链加 || funct7==7'b0001000

第 10 题 ROR

- **alu.v:** case 加 OP_ROR : result = (a >> b[4:0]) | (a << (32 - b[4:0]));
- **alu_ctrl.v:** 加 7'b0001001: alu_op = OP_ROR;
- **ctrl.v:** if 链加 || funct7==7'b0001001

第 11 题 CLZ

- **alu.v:** case 加 (嵌套三元, 从高位查第一个 1) `OP_CLZ : result = a[31]?0 : a[30]?1 : a[29]?2 : ... : a[1]?30 : a[0]?31 : 32;` (32 个分支, 可现场用脚本/手写全 32 位; 全 0 返回 32)
- **alu_ctrl.v:** 加 `7'b0001010: alu_op = OP_CLZ;`
- **ctrl.v:** if 链加 `|| funct7==7'b0001010`

第 12 题 SEQZ

- **alu.v:** case 加 `OP_SEQZ: result = (a == 32'b0) ? 32'd1 : 32'd0;`
- **alu_ctrl.v:** 加 `7'b0001011: alu_op = OP_SEQZ;`
- **ctrl.v:** if 链加 `|| funct7==7'b0001011`

Part B: 考试现场流程手册

考试场景: 教师现场给出新指令的格式 / opcode / 32 位机器码 + 测试样例。你需要: ① 改 RTL; ② 写 asm; ③ 用 RARS 生成 txt 机器码; ④ 烧 bit; ⑤ 用 Ddftest 跑用例。**核心契约** (不可改): 用例编号 0 写到 `0x4000`, 操作数写 `0x4004 / 0x4008`, 结果存 `0x400C`。

⚡ 开考第一件事 (最重要, 别忘)

把 `sw[15]` 拨到 1 → 单周期模式 (cpu_top 里 `mode_select=sw[15]`, =1 选单周期核)。

为什么: 项目里有单周期核 (cpu.v) 和流水核 (cpu_pipe.v) 两个, 靠 `sw[15]` 切换。两个核**共用**同一套 leaf 模块 (`alu.v / alu_ctrl.v / ctrl.v / imm_gen.v / load_store_fmt.v`) —— **改 leaf 模块, 两个核自动都生效。**

所以单周期模式下加新指令: - R/I 算术、Load、Store、新 opcode、新立即数 → **只改 leaf 模块** - 新分支 → leaf + **cpu.v** 的 **take_branch** (单周期那份) - **全程不碰 cpu_pipe.v** (本手册凡提到改 `cpu_pipe.v` 的, 单周期模式一律跳过)

这样最省事、最不会错。**切到流水模式才需要改 cpu_pipe.v, 考试不用。**

第 0 部分：考前 30 分钟必做（带去考场的"自检流水线"）

0.1 资料下载清单（断网前完成，全部存 D 盘）

资料	来源	路径建议
Project 代码（含已改好的 cpu_project）	GitHub / BB	D:\backup\cpu_project\
RARS jar	课程网站 / 网盘	D:\backup\rars.jar
Difftest 工具	课程网站	D:\backup\Difftest\
本复习手册	U 盘 / 邮箱	D:\backup\复习手册.md
EGo1 user manual	课程资料	D:\backup\EGo1_UserManual.pdf
改指令的笔记（自己整理的）	自己	D:\backup\笔记.md

0.2 自检流水线（断网前跑一次）

1. Vivado 打开 cpu_project → Run Synthesis → 通过
2. Run Implementation → 生成 bitstream → 通过
3. Open Hardware Manager → 烧到 EGo1 板 → 成功
4. Difftest → 选 COM 口 → Connect → ping → 收到 pong
5. CLI 发: halt → reset → 烧入一段 ADD 测试程序 → 跑 → 读 reg 1 验证

自检通过后再断网。任何一步不通，立刻 debug，不要拖到考试。

第 1 部分：考试题目格式预测（教师可能出的所有类型）

按教师可能给出的 "AAA s0,a0,a1（实现某种特定功能）" 格式，所有可能的指令类型就这 6 种：

类型	例子	编码格式	必改文件数
R-Type 算术	MIN s0, a0, a1	R	3 个
I-Type 算术	MAXI s0, a0, imm	I	1-3 个
I-Type Load	LBX s0, imm(a0)	I	1 个
S-Type Store	SWN a1, imm(a0)	S	1-3 个
B-Type Branch	BLE a0, a1, imm	B	2-3 个
U/J-Type	LUIN s0, imm 等	U/J	1-2 个

90% 概率出 R-Type 或 I-Type 算术（最容易出题、最容易当场验证）。10% 概率出 Load/Store/Branch（需要内存或控制流配合）。

第 2 部分：6 种类型的"完整改法"（按文件 + 行号）

路径一览（单周期模式）

文件	作用	两核共享？
ctrl.v	主控制信号	✓ 共享（改一次两核生效）
alu_ctrl.v	opcode → alu_op 译码	✓ 共享
alu.v	实际运算	✓ 共享
imm_gen.v	立即数拼接	✓ 共享
load_store_fmt.v	访存字节格式	✓ 共享
cpu.v	单周期 take_branch（仅新分支用）	仅单周期
~~ cpu_pipe.v ~~	流水 take_branch / hazard	单周期模式不用碰

关键：上面 5 个 leaf 模块单周期和流水**共用同一份**，改它们两核自动生效。单周期模式只有"新分支"才额外改 cpu.v 一处；cpu_pipe.v 全程不动。

文件路径：D:\Projects\CS202-project\cpu_project\cpu_project.srscs\sources_1\cpu\

类型 1：R-Type 算术（rd = rs1 OP rs2）

例子

- MIN s0, a0, a1 → $s0 = \min(a0, a1)$ （有符号取小）
- MAX s0, a0, a1 → 取大
- XNOR s0, a0, a1 → $\sim(a0 \wedge a1)$
- NAND s0, a0, a1 → $\sim(a0 \& a1)$
- ABS_DIFF s0, a0, a1 → $|a0 - a1|$

改 3 个文件

🔧 文件 1：alu.v

第 26 行附近（预留编码区）选一个未用的编码：

```
localparam OP_NEW = 4'b1010;    // 可用：1010 / 1100 / 1101 / 1110 / 1111
```

第 33 行 case (alu_op) 里加一行，根据题目函数定义写运算：

```

OP_NEW : result = (sa < sb) ? a : b;          // MIN (有符号取小)
// 或者
OP_NEW : result = (sa > sb) ? a : b;          // MAX
// 或者
OP_NEW : result = ~(a ^ b);                  // XNOR
// 或者
OP_NEW : result = ~(a & b);                  // NAND
// 或者: a 减 b 取绝对值
OP_NEW : result = (sa < sb) ? (b - a) : (a - b); // ABS_DIFF

```

文件 2: alu_ctrl.v

情况 A: 题目给的 opcode = 0110011 (复用 RV32I R-Type opcode)

第 31-40 行的 funct3=000 funct7 case 里加 (前提是题目 funct3=000) :

```

case (funct7)
    7'b00000000: alu_op = OP_ADD;
    7'b01000000: alu_op = OP_SUB;
    7'b00000001: alu_op = OP_MUL;
    7'b<题目funct7>: alu_op = OP_NEW; // ← 加这一行
    default:      alu_op = OP_ADD;
endcase

```

或者, 题目用了没用过的 funct3 (如 011), 第 30 行的外层 case 里加:

```

3'b<题目funct3>: alu_op = OP_NEW; // ← 加这一行

```

情况 B: 题目给的 opcode 是新值 (如 0001011)

在 alu_ctrl.v 第 26 行 case (opcode) 里加新 case (仿照 0110011 的写法) :

```

7'b<题目opcode>: begin
    case (funct3)
        3'b<题目funct3>: alu_op = OP_NEW;
        default: alu_op = OP_ADD;
    endcase
end

```

文件 3: ctrl.v

情况 A: 题目 opcode = 0110011

不用改 ctrl.v! 现有 R-Type case (第 51-85 行) 已经覆盖所有 funct3, 会自动设好 reg_write=1。

情况 B: 题目 opcode 是新值

在 ctrl.v 第 50 行 case (opcode) 里加新 case:


```

7'b<题目opcode>: begin    // 新 R-Type
    alu_src_a = 2'b00; alu_src_b = 1'b0;
    wb_src     = 2'b00; reg_write = 1'b1;
end

```

单周期模式 (sw[15]=1) 不用改 cpu_pipe.v。 下面这步只在切流水模式时才需要：流水核要让 hazard 检测知道新 opcode 读 rs1/rs2，否则前递/stall 可能出错——在 cpu_pipe.v 的 id_uses_rs1/id_uses_rs2 case 加 7'b<题目opcode>: begin id_uses_rs1=1; id_uses_rs2=1; end。考试演示单周期 → 跳过本步。

测试 asm 模板 (R-Type)

```

# 题目: NEW_OP rd, rs1, rs2
# 基址 gp = 0x4000, case 编号 0, 结果存 0x400C

main:
    # ---- 1. 从 0x4000 读用例编号、操作数 ----
    lw    t0, 0(gp)        # t0 = 用例编号 (0)
    lw    a0, 4(gp)        # a0 = 操作数 A
    lw    a1, 8(gp)        # a1 = 操作数 B

    # ---- 2. 执行新指令 (先用 add 占位, 后续替换机器码) ----
    add    s0, a0, a1      # ★ 占位符, 把这条机器码替换成新指令机器码

    # ---- 3. 把结果存回 0x400C ----
    sw     s0, 12(gp)

done:
    jal    zero, done      # 死循环

```

注意 RARS 生成的机器码里： add s0, a0, a1 = 0x00B50433，你要找到这个 hex 然后替换成老师给的机器码。

类型 2: I-Type 算术 (rd = rs1 OP imm)

例子

- MAXI s0, a0, 0xFF → s0 = max(a0, 0xFF)
- ABSI s0, a0 → s0 = abs(a0)
- ROTRI s0, a0, 4 → 循环右移 4 位
- CLZI s0, a0 → leading-zero count

改 1-3 个文件

🔧 文件 1: `alu.v` (同 R-Type, 加一个新运算)

例: 循环右移

```
localparam OP_R0TR = 4'b1010;  
// case 里加  
OP_R0TR: result = (a >> shamt) | (a << (32 - shamt));
```

🔧 文件 2: `alu_ctrl.v`

情况 A: 题目 `opcode = 0010011` (复用 I-ALU `opcode`)

第 55 行 `case (funct3)` 里加:

```
3'b<题目funct3>: alu_op = OP_R0TR;          // ← 加
```

如果用 `funct3=001/101` (移位类), 需要在 `alu_ctrl.v` 第 61 行像 SRL/SRA 那样用 **funct7 区分**:

```
3'b101: begin  
    case (funct7)  
        7'b0000000: alu_op = OP_SRL;  
        7'b0100000: alu_op = OP_SRA;  
        7'b<题目funct7>: alu_op = OP_R0TR;  
        default:    alu_op = OP_SRL;  
    endcase  
end
```

🔧 文件 3: `ctrl.v`

情况 A: `opcode = 0010011`, `funct3` 不是 `001/101`: 不用改! 第 110 行 `default` 已经处理了。

情况 A 但 `funct3 = 001/101`: 第 93-109 行加 `funct7` 判断:

```
3'b101: begin  
    if (funct7 == FUNCT7_STD || funct7 == FUNCT7_ALT || funct7 == 7'b<题目funct7>) begin  
        alu_src_a = 2'b00; alu_src_b = 1'b1;  
        wb_src    = 2'b00; reg_write = 1'b1;  
        imm_type  = 3'b000;  
    end  
end
```

情况 B: 题目 `opcode` 是新值: 在 `ctrl.v` 加新 `case` (仿照 `7'b0010011`)。

测试 asm 模板 (I-Type)

```
main:
    lw    t0, 0(gp)          # case id (0)
    lw    a0, 4(gp)          # 操作数 A
    # operand B 不用 (或当成 immediate 由汇编指定)

    addi s0, a0, 5           # ★ 占位符 (imm=5), 替换为新指令机器码

    sw    s0, 12(gp)

done:
    jal   zero, done
```

占位用 ADDI 时, imm 值就用题目里的立即数。 `addi s0, a0, 5` = `0x00550413`。

类型 3: I-Type Load (rd = MEM[rs1+imm], 自定义版本)

例子

- `LBX s0, 0(a0)` → 同 LB (符号扩展取字节), 但用新 opcode

改 1 个文件

🔧 文件: `ctrl.v` 第 117-129 行后插入

```
7'b<题目opcode>: begin // 新 Load
    case (funct3)
        3'b<题目funct3>: begin
            alu_src_a = 2'b00; alu_src_b = 1'b1;
            wb_src    = 2'b01; reg_write = 1'b1;
            mem_read  = 1'b1; imm_type  = 3'b000;
        end
        default: ;
    endcase
end
```

单周期模式跳过。 仅切流水模式时, `cpu_pipe.v` 的 `id_uses` 表加 `7'b<题目opcode>: begin id_uses_rs1=1; id_uses_rs2=0; end`。

测试 asm 模板 (Load)

```
main:
    lw    t0, 0(gp)           # case id
    lw    a0, 4(gp)           # 内存地址 (base)
    # B 不用

    sw    a0, 12(gp)           # 占位 (先把数据写到 0x400C 的备份)
    # 实际测试要先存数据再 load:
    li    t1, 0xAB
    sb    t1, 0(a0)           # 把 0xAB 存到 [a0]

    lb    s0, 0(a0)           # ★ 占位符, 替换成 LBX 机器码

    sw    s0, 12(gp)

done:
    jal   zero, done
```

类型 4: S-Type Store ($\text{MEM}[\text{rs1}+\text{imm}] = f(\text{rs2})$)

例子

- `SWN a1, 0(a0)` $\rightarrow \text{MEM}[\text{a0}+\text{imm}] = -\text{a1}$ (取负后存)

改 1-2 个文件

🔧 文件 1: `ctrl.v` 第 130-139 行后加:

```
7'b<题目opcode>: begin    // 新 Store
    case (funct3)
        3'b<题目funct3>: begin
            alu_src_a = 2'b00; alu_src_b = 1'b1;
            mem_write = 1'b1; imm_type = 3'b001;
        end
    endcase
end
```

🔧 文件 2 (可能要): 如果题目要求"对 rs2 做变换再存" (如 SWN 取负), 需要在 `cpu.v` 第 136 行附近改 `load_store_fmt` 的输入 `rs2_data`:

```

wire [31:0] rs2_for_store = is_swn ? (~rs2_data + 1) : rs2_data;
load_store_fmt u_lsfmt (
    ...
    .rs2_data(rs2_for_store),
    ...
);

```

注意：这种"带变换的 Store"是题目里**最难**的类型之一，需要从 ctrl.v 输出一个 is_swn 信号传给 cpu.v。**简单做法：**直接在 cpu.v 里检测 opcode == 题目 opcode。

测试 asm 模板 (Store)

```

main:
    lw    t0, 0(gp)
    lw    a0, 4(gp)      # 操作数 A (要存的值)
    # 需要一个目标地址 → 直接用 gp+12 = 0x400C

    sw    a0, 12(gp)      # ★ 占位符 (先用 sw 占位)

done:
    jal   zero, done

```

如果是 SWN 这种"先变换再存"，期望结果就是 -a0，**测试时会自然跑出结果。**

类型 5: B-Type Branch (条件分支)

例子

- BLE a0, a1, imm → if (a0 ≤ a1) PC += imm
- BNZ rs1, imm → if (rs1 != 0) jump

改 2-3 个文件

 **文件 1: ctrl.v** 第 140-155 行的 Branch case，加新 funct3:

```

case (funct3)
    3'b000, 3'b001, 3'b100, 3'b101, 3'b110, 3'b111,
    3'b<题目funct3>: begin                                // ← 加
        alu_src_a = 2'b00; alu_src_b = 1'b0;
        branch    = 1'b1; imm_type  = 3'b010;
    end
endcase

```

🔧 文件 2: `alu_ctrl.v` 第 72-77 行 Branch case 加:

```
case (funct3)
  3'b000, 3'b001: alu_op = OP_SUB;          // BEQ/BNE
  3'b100, 3'b101: alu_op = OP_SLT;          // BLT/BGE
  3'b110, 3'b111: alu_op = OP_SLTU;         // BLTU/BGEU
  3'b<题目funct3>: alu_op = OP_SLT;         // ← 例: BLE 用 SLT
  default: alu_op = OP_SUB;
endcase
```

🔧 文件 3: `cpu.v` 加 `take_branch` 判定 (单周期只改这一处)。

`cpu.v` 第 120-131 行:

```
case (funct3)
  3'b000: take_branch = alu_zero;
  3'b001: take_branch = ~alu_zero;
  3'b100: take_branch = alu_result[0];
  3'b101: take_branch = ~alu_result[0];
  3'b110: take_branch = alu_result[0];
  3'b111: take_branch = ~alu_result[0];
  3'b<题目funct3>: take_branch = alu_result[0] | alu_zero; // ← 例: BLE
  default: take_branch = 1'b0;
endcase
```

单周期模式不用改 `cpu_pipe.v`。 切流水模式才需要在 `cpu_pipe.v` 的 `ex_take_branch` case 加同样一行。

测试 asm 模板 (Branch)

```
main:
    lw    t0, 0(gp)
    lw    a0, 4(gp)
    lw    a1, 8(gp)

    li    s0, 0           # 默认 0
    blt   a0, a1, take    # * 占位符, 替换成新分支机器码 (题目 funct3)
    li    s0, 99          # 没跳到则赋值 99
    jal   zero, store
take:
    li    s0, 42          # 跳到则赋值 42
store:
    sw    s0, 12(gp)

done:
    jal   zero, done
```

测试时操作数 (a0, a1) 决定跳不跳, 期望值 42 或 99 也由此决定。

类型 6: U-Type / J-Type

例子

- `LUI s0, 0x12345` → `s0 = -(imm << 12)`
- `JX rd, imm` → 类似 JAL 但有特殊行为

改 1-2 个文件

 `ctrl.v` 加新 opcode case (仿照 LUI 第 175 行 / JAL 第 156 行) :

LUI 类 (U-Type) :

```
7'b<题目opcode>: begin
    alu_src_a = 2'b10; alu_src_b = 1'b1;
    wb_src    = 2'b00; reg_write = 1'b1;
    imm_type   = 3'b011;
end
```

如果要"取负", 把 `alu_op` 改成 `OP_SUB` (让 ALU 算 `0 - imm`) : - 这种情况还要改 `alu_ctrl.v`: `7'b<题目opcode>: alu_op = OP_SUB;`

测试 asm 模板 (U-Type)

```
main:
    lw    t0, 0(gp)

    lui   s0, 0x12345        # ★ 占位符 (imm=0x12345), 替换成 LUI 机器码

    sw    s0, 12(gp)

done:
    jal   zero, done
```

类型 7: 全新立即数格式 (imm_gen 也要改, 少见但要会)

什么时候遇到

新指令的立即数**位段排布**和现有 5 种格式 (I/S/B/U/J) 都不一样时。例如: - `LI16 rd, imm16` → `rd = {16'b0, inst[31:16]}` (16 位立即数放高半字, 零扩展)

现有格式都拼不出 "inst[31:16] 零扩展", 所以要在 imm_gen 加一种新格式。

改 2 个文件

🔧 文件 1: `imm_gen.v`

加一根新的立即数拼接 + case 分支 (用一个没用过的 imm_type 编码, 如 101):

```
wire [31:0] imm_x = {16'b0, inst[31:16]};    // 按题目的位段拼, 符号位看是否扩展
// case 里加:
3'b101:  imm = imm_x;
```

拼接口诀: 高位用 `{N{inst[31]}}` 做符号扩展 (无符号就补 0), 低位按题目摆字段。立即数位段不能和 opcode(inst[6:0])、rd(inst[11:7]) 重叠——它们是固定字段。

🔧 文件 2: `ctrl.v`

加新 opcode case, imm_type 设成你刚加的新编码:

```
7'b<题目opcode>: begin
    alu_src_a = 2'b10; alu_src_b = 1'b1;    // ZERO + imm (rd=0+imm=imm)
    wb_src    = 2'b00; reg_write = 1'b1;
    imm_type   = 3'b101;                    // ← 用新格式
end
```

alu_ctrl 默认 ADD 即可 (0+imm=imm), 不用改。

测试 asm 模板

立即数格式特殊，**RARS 可能没有这条指令**，直接手算机器码，用 `add s0,a0,a1` 占位再替换。

第 3 部分：手算机器码 5 步法（必背）

6 种格式速查

R-Type:	funct7	rs2	rs1	funct3	rd	opcode	
	7	5	5	3	5	7	
I-Type:	imm[11:0]	rs1	funct3	rd	opcode		
	12	5	3	5	7		
S-Type:	imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode	
	7	5	5	3	5	7	
B-Type:	imm12	imm[10:5]	rs2	rs1	funct3	imm[4:1]	imm11 opcode
U-Type:	imm[31:12]				rd	opcode	
J-Type:	imm20	imm[10:1]	imm11	imm[19:12]	rd	opcode	

寄存器编号速查

x0 = 00000 zero	x16 = 10000 a6
x1 = 00001 ra	x17 = 10001 a7
x2 = 00010 sp	x18 = 10010 s2
x3 = 00011 gp	...
x4 = 00100 tp	x28 = 11100 t3
x5 = 00101 t0	x29 = 11101 t4
x6 = 00110 t1	x30 = 11110 t5
x7 = 00111 t2	x31 = 11111 t6
x8 = 01000 s0/fp	★ 题目 s0 是 x8
x9 = 01001 s1	
x10 = 01010 a0	★ 题目 a0 是 x10
x11 = 01011 a1	★ 题目 a1 是 x11
x12 = 01100 a2	

5 步法（用 `MIN s0, a0, a1` 演示）

题目设：opcode=0001011, funct3=000, funct7=0000010

Step 1: 写格式

```
R-Type: funct7 | rs2 | rs1 | funct3 | rd | opcode
```

Step 2: 填二进制

```
funct7 = 0000010
rs2    = a1 = 01011
rs1    = a0 = 01010
funct3 = 000
rd      = s0 = 01000
opcode = 0001011
```

Step 3: 拼接

```
0000010 01011 01010 000 01000 0001011
```

Step 4: 每 4 位切一段

```
0000 0100 1011 0101 0000 0010 0000 1011
```

Step 5: 转 hex

```
0    4    B    5    0    2    0    B
→ 0x04B5020B
```

负数立即数手算

-127 的 12 位补码: 1. $+127 = 0x07F = 000001111111$ 2. 取反 = 111110000000 3. 加 1 = $111110000001 = 0xF81$

记忆法: 12 位负数补码 = $0x1000 - |n| - 1 = 0xFFF - 127 = 0xF81 - 2048 = 0x800$

第 4 部分: 考试现场完整 workflow (按时间顺序)

阶段 0: 开考前 5 分钟

- 读题, 抄下: 指令名、opcode、funct3/funct7、imm 格式、机器码示例
- 判断指令类型 (参考第 1 部分表格)

阶段 1: 写 asm (5-10 min)

打开 RARS, 新建文件 `inst.asm`, 按第 2 部分的对应模板写。

标准框架:

```
.text
main:
    lw    t0, 0(gp)        # case id
    lw    a0, 4(gp)        # operand A
    lw    a1, 8(gp)        # operand B

    # ★ 占位符指令，记住 RARS 生成的机器码 hex
    add   s0, a0, a1

    sw    s0, 12(gp)

done:
    jal   zero, done
```

gp 的初始值：Difftest 框架会保证 `gp = 0x4000` (hardwired)。你的代码不需要 `li gp, 0x4000`。

阶段 2：RARS 生成机器码 (2 min)

1. RARS 菜单 Run → Assemble (F3)
2. File → Dump Memory
3. Memory Segment: `.text (0x00400000-0x00400xxx)`
4. Format: Hexadecimal Text
5. Dump To File... → 保存为 `inst.txt`

RARS 默认 `.text` 起始地址 `0x00400000`，但你的 IMem 起始 `0x0`。**没关系**——dump 出的是按顺序的机器码，每行一条，按 wi 顺序写到 `0x00, 0x04, 0x08...` 即可。

阶段 3：替换占位符 (1 min)

打开 `inst.txt`，找到 `add s0, a0, a1` 对应的 hex `0x00B50433`，**替换成老师给的机器码：**

```
0x0001A283 # lw t0, 0(gp)          ← gp=0x4000, rs1=gp, imm=0
0x0041A503 # lw a0, 4(gp)
0x0081A583 # lw a1, 8(gp)
0x00B50433 # ★ 替换这行 → 老师的机器码 (add s0,a0,a1 占位)
0x0081A623 # sw s0, 12(gp)
0x0000006F # jal zero, done (PC 自跳)
```

上面的机器码已手算验证 (`gp=x3`)。**实际仍以你 RARS dump 的为准**——寄存器/立即数不同，机器码就不同，别照抄。

阶段 4: 改 RTL (20-40 min)

按第 2 部分对应类型的"改法", 改 `ctrl.v` / `alu_ctrl.v` / `alu.v` (必要时 `cpu.v`) 。

保存所有文件!

阶段 5: 把 inst.txt 烧进 IMem (两条路径, 选一)

路径 A: 通过 ifetch.v INIT_FILE 静态初始化 (推荐)

1. 把 `inst.txt` 复制到 `D:\Projects\CS202-project\cpu_project\`
2. 打开 `TopDebug.v`, 找到 `u_ifetch` 例化 (约第 363 行), 把 `INIT_FILE` 从空改成文件名: `verilog ifetch #(.INIT_FILE("")) u_ifetch ( // 原来 ifetch #(.INIT_FILE("inst.txt")) u_ifetch ( // 改成这样`

`ifetch.v` 内部 `if (INIT_FILE != "") $readmemh(INIT_FILE, mem);`, 所以填了文件名就会自动加载。

3. 把 `inst.txt` 添加到 Vivado 工程: - Sources 面板 → 右键 → Add Sources → Add Files → 选 `inst.txt` → 勾选 "Copy sources into project"
4. 综合 → 实现 → 生成 bit → 烧板

优点: 上电就跑, 不用每次 wi。缺点: 换测试要重新综合。

路径 B: 通过 Difftest CLI wi 命令烧 (最稳)

1. 综合 → 烧 bit (不改 `INIT_FILE`)
2. Difftest CLI: ``

`halt reset wi 0x00 0x[第1行] wi 0x04 0x[第2行] wi 0x08 0x[第3行] wi 0x0C 0x[第4行] ← 老师指定的机器码 wi 0x10 0x[第5行] wi 0x14 0x[第6行] ```

优点: 换测试不用重综合, 速度快。缺点: 每次都要敲命令 (可批量粘贴) 。

阶段 6: 用 Difftest 跑测试用例 (核心)

按教师给的测试样例:

```
0, [操作数A1, 操作数B1], 期望结果1
0, [操作数A2, 操作数B2], 期望结果2
```

对每条样例:

```
=== 写测试输入到 DMem ===
> halt
> reset
> wd 0x4000 0x00000000      # case id = 0
> wd 0x4004 0x[操作数A]     # A
> wd 0x4008 0x[操作数B]     # B
> wd 0x400C 0xDEADBEEF      # 故意写垃圾值，验证 CPU 真的覆盖了

=== 跑 ===
> reset                      # 让 PC 归 0
> run
    (等 0.5-1 秒)
> halt

=== 验证 ===
> rd 0x400C
< 0x[期望结果]              ← 等于这个就 PASS
```

对每条用例重复以上流程。

阶段 7：跑通了 → 喊老师验收

未跑通的排查见第 5 部分。

第 5 部分：出错排查表（金子般重要）

5.1 综合阶段出错

错误	原因	解决
multiple drivers	alu_op 在多个 case 都 assign	检查只在一个分支赋值
localparam redeclared	同一名字的 OP_xxx 定义两次	合并
case item duplicate	funct3/funct7 重复	改成不同值
Synthesis 卡住或综合超慢	错误综合循环	重启 Vivado

5.2 烧板后 ping 不通

```
重烧 bitstream → 按 EGo1 板上 S6 reset → 重连 Diffptest
仍不通：
- 检查 COM 口（设备管理器）
- 波特率必须 115200
- USB 线接触不良？
```

5.3 CLI 有响应但结果不对

排查思路：先看 IMem，再看寄存器，再看 PC

```
=== 看 IMem 烧对了吗 ===
> ri 0x0C
< 0x[第4条]    ← 应等于老师机器码

=== 看 PC 跑到哪 ===
> pc
< 0x[死循环地址]    ← 应等于 jal zero, done 的地址

=== 看寄存器 ===
> reg 8                # s0
< 0x[期望值]
> reg 10               # a0
< 0x[操作数A]
> reg 11               # a1
< 0x[操作数B]
```

5.4 单步 debug

```
> halt
> reset
> step
> pc                # 应是 0x04
> reg 5             # t0 应等于 case id
> step
> pc                # 应是 0x08
> reg 10            # a0 应等于 0x4004 处的值
... 一步步走
```

哪一步后某寄存器不对，那条指令就出错了。

5.5 常见症状

症状	可能原因	解决
reg s0 = 0	reg_write 没置 1	检查 ctrl.v 加的 case
reg s0 等于 ALU 算了但不是新运算	alu_op 译码没走新分支	检查 alu_ctrl.v
程序跑飞 (PC 远离烧入区)	末尾 jal 没烧对 / 跳转目标算错	重新检查 inst.txt 末尾
0x400C 还是 0xDEADBEEF	sw 没执行 / mem_write 没置 1	检查 ctrl.v Store case; step 看是否走到 sw
数值高位 24 位错 (应该 FFFFFFFF 但是 0)	符号扩展没对	检查 funct3 → mem_signed 的映射
单周期对但流水模式跑不出	hazard 漏了新 opcode	切回单周期(sw[15]=1)即可; 非要流水则改 cpu_pipe.v 的 id_uses 表

第 6 部分：Difftest CLI 命令速查（记忆卡）

连通 / 控制

ping	测连通
halt	停 CPU
reset	PC 归 0（不清 IMem/DMem）
run	启动
step	单步（必须先 halt）

读取

pc	当前 PC
reg <N>	寄存器 xN（也可以 reg s0 / reg a0）
ri <addr>	读 IMem
rd <addr>	读 DMem

写入

wi <addr> <data>	写 IMem
wd <addr> <data>	写 DMem

万能流程

```
ping → halt → reset
→ wi×N（烧指令）[或路径 A 已烧入]
→ ri 验证
→ wd 0x4000 0/4/8（写 case id + 操作数）
→ reset → run → 等 → halt
→ rd 0x400C 看结果
```

第 7 部分：现场完整模板示例 — R-Type MIN

假设老师给：- 名称：MIN s0, a0, a1 - 功能：s0 = min(a0, a1) 有符号 - 编码：opcode=0001011, funct3=000, funct7=0000010 - 测试：0, [7, 3], 3、0, [-5, 2], -5 (0xFFFFFFFFB)

Step 1：判断类型 → R-Type 算术（opcode 是新值，情况 B）

Step 2：手算机器码

MIN s0(=x8), a0(=x10), a1(=x11)：


```
funct7=0000010 rs2=a1=01011 rs1=a0=01010 funct3=000 rd=s0=01000 opcode=0001011
拼接: 0000010 01011 01010 000 01000 0001011
→ 0000 0100 1011 0101 0000 0100 0000 1011
→ 0x04B5040B
```

注意 rd=01000 拼到 [11:7] 位置: ... funct3(000) | rd(01000) | opcode(0001011) ... = ...
000 01000 0001011

Step 3: 写 asm

```
.text
main:
    lw    t0, 0(gp)        # case id
    lw    a0, 4(gp)        # operand A
    lw    a1, 8(gp)        # operand B

    add   s0, a0, a1        # ★ 占位符 (机器码 0x00B50433)

    sw    s0, 12(gp)       # 结果写 0x400C

done:
    jal   zero, done
```

Step 4: RARS 生成 inst.txt (替换前)

```
0x0001A283 # lw t0, 0(gp)
0x0041A503 # lw a0, 4(gp)
0x0081A583 # lw a1, 8(gp)
0x00B50433 # ★ add s0, a0, a1 (要替换)
0x0081A623 # sw s0, 12(gp)
0x0000006F # jal zero, 0
```

Step 5: 替换占位符

把第 4 行 0x00B50433 改成 0x04B5040B (MIN 机器码)。

Step 6: 改 RTL (情况 B: 新 opcode)

alu.v 第 26 行后加:

```
localparam OP_MIN = 4'b1010;
// case 里加:
OP_MIN: result = (sa < sb) ? a : b;
```

alu_ctrl.v 第 26 行 case (opcode) 末尾 default 之前加:

```
7'b0001011: begin
    case (funct3)
        3'b000: alu_op = OP_MIN;
        default: alu_op = OP_ADD;
    endcase
end
```

ctrl.v 第 50 行 case (opcode) 里加（在 default 之前）：

```
7'b0001011: begin
    alu_src_a = 2'b00; alu_src_b = 1'b0;
    wb_src    = 2'b00; reg_write = 1'b1;
end
```

单周期模式（sw[15]=1）到此为止，不用改 cpu_pipe.v。 上面 3 个文件（alu/alu_ctrl/ctrl）改完就能跑。只有流水线模式才需要在 cpu_pipe.v 的 id_uses 表加 7'b0001011: begin id_uses_rs1=1; id_uses_rs2=1; end。

Step 7: 综合 + 烧板

Step 8: Difftest CLI 测试

```
> ping                                < pong

# === 烧指令 (路径 B) ===
> halt                                < ack
> reset                               < ack
> wi 0x00 0x0001A283
> wi 0x04 0x0041A503
> wi 0x08 0x0081A583
> wi 0x0C 0x04B5040B                  ← MIN
> wi 0x10 0x0081A623
> wi 0x14 0x0000006F

# === 用例 1: [7, 3] → 期望 3 ===
> wd 0x4000 0x00000000
> wd 0x4004 0x00000007
> wd 0x4008 0x00000003
> wd 0x400C 0xDEADBEEF
> reset                               < ack
> run                                 < ack
    (等 0.5 秒)
> halt                                < ack
> rd 0x400C
< 0x00000003                          ✓ PASS

# === 用例 2: [-5, 2] → 期望 -5 = 0xFFFFFFF B ===
> wd 0x4004 0xFFFFFFF B
> wd 0x4008 0x00000002
> wd 0x400C 0xDEADBEEF
> reset
> run
    (等)
> halt
> rd 0x400C
< 0xFFFFFFF B                        ✓ PASS
```

Step 9: 喊老师验收

第 8 部分：考试时间分配建议（120 分钟）

阶段	时间	任务
0-10 min	读题 + 判断类型 + 手算机器码	
10-20 min	写 asm + RARS 生成 inst.txt + 替换	
20-50 min	改 RTL（按文件顺序：alu.v → alu_ctrl.v → ctrl.v）	
50-70 min	Vivado 综合 + 烧 bit	
70-90 min	Difftest CLI 跑测试用例	
90-120 min	Debug / 排查（如果出错）	

20 min 缓冲建议留出来：综合卡住、烧板失败、wi 错位等都很常见。

第 9 部分：关键路径文件位置（收藏）

RTL 改动：

D:\Projects\CS202-project\cpu_project\cpu_project.srscs\sources_1\cpu\

- ├─ ctrl.v ← 主控制信号 (共享, 两核生效)
- ├─ alu_ctrl.v ← opcode → alu_op (共享)
- ├─ alu.v ← 实际运算 (共享)
- ├─ imm_gen.v ← 立即数拼接 (共享)
- ├─ load_store_fmt.v ← 访存字节格式 (共享)
- ├─ cpu.v ← 单周期 take_branch (单周期才改)
- └─ cpu_pipe.v ← 流水线 (单周期模式不碰!)

工程文件：

D:\Projects\CS202-project\cpu_project\

- ├─ cpu_project.xpr ← Vivado 工程
- └─ inst.txt ← 测试程序机器码（需自己创建）

XDC：

D:\Projects\CS202-project\cpu_project\cpu_project.srscs\constrs_1\

- └─ top.xdc

参考资料：

D:\Projects\CS202-project\

- ├─ assembly\src_asm\batch_test.asm ← 参考 case 分发器写法
- ├─ assembly\machine_code\batch_test.txt ← 现成机器码可参考
- ├─ 接口对齐检查单.md ← 接口契约
- └─ Verification Tutorial.pdf ← Difftest 工具手册

祝你考试顺利！

最后的话：考试现场最容易出错的不是 RTL 逻辑，而是**手算机器码的位错位 + inst.txt 替换错位**。务必：

1. 手算后**反推一遍**验证（拆 32 位回原字段）
2. inst.txt 替换前**比对行号**（占位符在第几行就改第几行）
3. 烧入后**先 ri 回读验证**再 run
4. 跑出错先 **step + pc** 不要瞎改代码

祝大胜！